

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Teórica 5: Tuplas, conjuntos y diccionarios

Repaso: listas

```
a = [11, 12, 13, 14]

len(a)          # cantidad de elementos

a.append(15)     # agregar al final

a[1]            # leer una posición

a[1] = 9999      # sobrescribir una posición

9999 in a        # pertenencia

a[1:3]          # obtener una sublista

for x in a:      # iterar la lista
    print(x)
```

Tupla

Las **tuplas** se usan para representar y almacenar múltiples valores en una única variable. Por ejemplo:

```
punto = (1.7, -0.9, 0.4)
```

(1.7, -0.9, 0.4) es una tupla formada por tres floats, y representa un punto en el espacio tridimensional.

En general:

- ▶ Los valores van entre paréntesis y separados por comas: (1, 2, 3)
- ▶ Una tupla puede tener una cantidad arbitraria de componentes: (1, 2), (1, 2, 3, 4), etc.
- ▶ Los valores pueden ser de distintos tipos: (0.1, 'hola', False)
- ▶ Las tupla, una vez creadas, no se puede agregar, modificar o borrar sus valores.

Ejemplos:

```
fecha = (25, 'mayo', 1810)  
persona = ('Juana', 'Fuláñez', 27)
```

Tupla (tuple)

Operaciones:

- ▶ `(x, y, z)` Crea y devuelve una tupla con tres valores.
- ▶ `len(t)` Devuelve la cantidad de componentes de la tupla `t`.
- ▶ `t[i]` Devuelve el valor de la i -ésima componente de `t`
(Requiere: $0 \leq i < \text{len}(t)$)
- ▶ `t == u` Compara `t` y `u`, componente a componente.

Tupla

Por favor, ¡no confundir tuplas y listas!

Una tupla `t` es solo una colección fija (**immutable**) de valores.

No podemos modificar una tupla:

- ▶ ~~`t.append(valor)`~~
- ▶ ~~`t[i] = valor`~~
- ▶ ~~`t.pop()`~~
- ▶ etc.

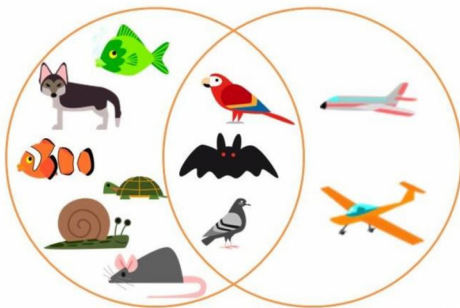
Lo más sencillo es pensar a una tupla como un par ordenado (una *dupla*), o una terna ordenada (una *tripla*), etc.

Conjunto

Hasta ahora trabajamos con `bool`, `int`, `float`, `string`, `list`.

Los tipos de datos son útiles para **representar datos**, y también para **escribir algoritmos más simples**.

¿Cómo hacemos para trabajar con conjuntos?



Conjunto (set)

Operaciones:

- ▶ `c = set()`: Crea un conjunto vacío.
- ▶ `c.add(x)`: Agrega el elemento `x` al conjunto `c`.
- ▶ `x in c`: Dice si el elemento `x` está en `c`.
- ▶ `c.remove(x)`: Elimina el elemento `x` de `c`.
Precondición: `x in c` (caso contrario, tira error)
- ▶ `len(c)`: Devuelve la cantidad de elementos de `c`.
- ▶ `c1==c2`: Dice si los dos conjuntos son iguales.
- ▶ `c1 | c2`: Devuelve un nuevo conjunto con $c1 \cup c2$.
- ▶ `c1 & c2`: Devuelve un nuevo conj. con $c1 \cap c2$.
- ▶ `c1 - c2`: Devuelve un nuevo conj. con $c1 \setminus c2$.
- ▶ `list(c)`: Devuelve una lista con los elementos de `c`.

Conjunto

Ejemplo:

```
felinos1 = set()
felinos1.add("león")
felinos1.add("gato")

felinos2 = set()
felinos2.add("gato")
felinos2.add("león")
felinos2.add("león")

felinos1 == felinos2      # Devuelve True

felinos1.add("tigre")
felinos2.add("puma")

felinos1 & felinos2
    # Devuelve {'gato', ' león'}

felinos1 | felinos2
    # Devuelve {'puma', 'gato', ' león', 'tigre'}
```


Conjunto

Ejercicio. (1)

- (a) Escribir una función `letras(palabra)`, que dada una palabra devuelva el conjunto de letras que la componen.

Ejemplo:

`letras('perezosos')  $\mapsto$  {'p', 'e', 'r', 'z', 'o', 's'}`

- (b) Escribir una función `letras_en_comun(pal1, pal2)` que dadas dos palabras, devuelva el conjunto de letras que tienen en común.

Ejemplo:

`letras_en_comun('pala', 'pelo')  $\mapsto$  {'p', 'l'}`

Nota: el orden en el que se muestran los elementos de un conjunto es irrelevante.

Diccionario (dict)

Asocia una **clave** (de algún tipo) a un **valor** (de algún tipo).

- ▶ Ej: palabra/definición, país/población, depto/inquilino.

Operaciones:

- ▶ `d = dict()`: Crea un diccionario vacío.
- ▶ `d[c] = v`: Asocia la clave `c` al valor `v` en el diccionario `d`.
- ▶ `c in d`: Dice si la clave `c` está definida en `d`.
- ▶ `d[c]`: Devuelve el valor asociado a la clave `c` en el dicc. `d`.
 Precondición: `c in d` (caso contrario, tira error)
- ▶ `d.keys()`: Devuelve las claves del diccionario.

Diccionario

Ejemplo:

```
peso_atómico = dict()

peso_atómico['H'] = 1.00797
peso_atómico['He'] = 4.0026
peso_atómico['C'] = 12.0111
peso_atómico['U'] = 238.02891

'Ag' in peso_atómico # devuelve False

peso_atómico['Ag'] = 107.8683
'Ag' in peso_atómico # devuelve True

peso_atómico.keys()
```

Diccionario

Ejercicio. (2) Escribir una función `digitos(numero)` que, dado un string que representa un número natural, devuelva un string enumerando sus dígitos.

Ejemplos:

`digitos('4137') ⟶ 'cuatro uno tres siete'`

`digitos('0') ⟶ 'cero'`

`digitos('11') ⟶ 'uno uno'`

Repaso de la clase de hoy

- ▶ Tuplas
- ▶ Conjuntos
- ▶ Diccionarios

Bibliografía complementaria:

- ▶ IPP, secciones 9.11 a 9.12, 11.1 y 12.1 a 12.3

Con lo visto, ya pueden resolver la sección 6 de la guía de ejercicios.